IMU vibration spectrum over MAVLink

How the Hybrid-PF firmware (ArduPilot `AP_HybridPF`, parameter `HPF_FFT_ENABLE`) computes the vibration spectrum of the IMU and packs it into the standard MAVLink message `DEBUG_FLOAT_ARRAY` (id 350), with a worked example of every step. Enough to decode the stream in any ground station, or to produce a compatible one on another autopilot.

The example signal is synthetic, with the timing of the real board: IMU reads at 880 to 950 Hz with jitter, a 0.02 rad/s vibration line at 81.25 Hz on the gyro X axis, a 0.3 m/s² line at 40 Hz on the accelerometer X axis, gravity on Z and white noise at the SCH16T-K10 datasheet density. It is computed with the same steps as `AP_HybridPF_Spectrum.cpp`. Section 8 shows real spectra and section 6 a real packet recorded from the CORV board.

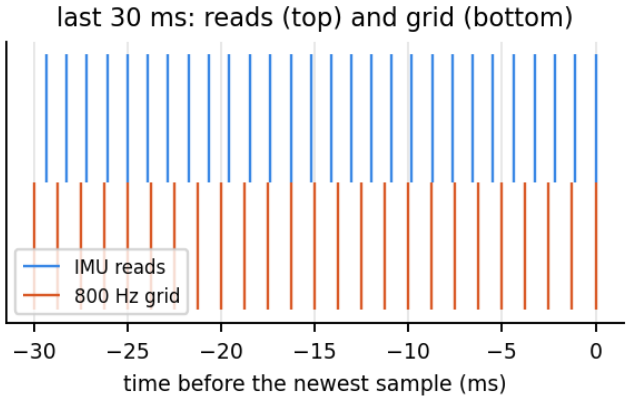
1. Parameters

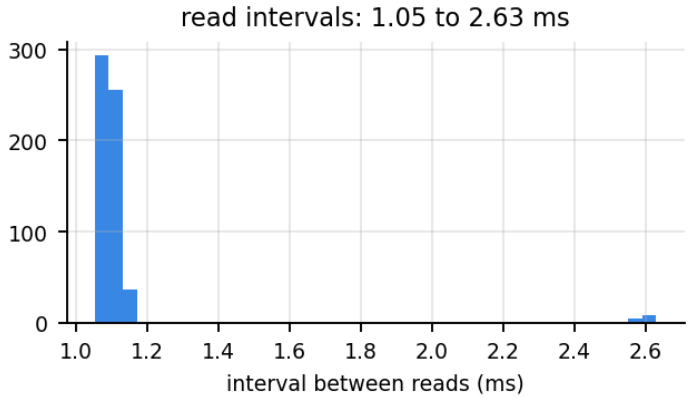
Quantity	Value	Note
Analysis rate fs	800 Hz	even grid the samples are resampled on, step 1250 µs
FFT length N	256	window length (N-1)/fs = 318.75 ms
Bin spacing df	fs / N = 3.125 Hz	
Bins sent	FFT bins 1 to 116	3.125 to 362.5 Hz; DC (bin 0) is not sent
Update	every 250 ms	about 4 spectra per second
Sensors	gyro, accelerometer	the three axes of each summed
Units	dB re 1 (rad/s)²/Hz, dB re 1 (m/s²)²/Hz	one-sided power spectral density

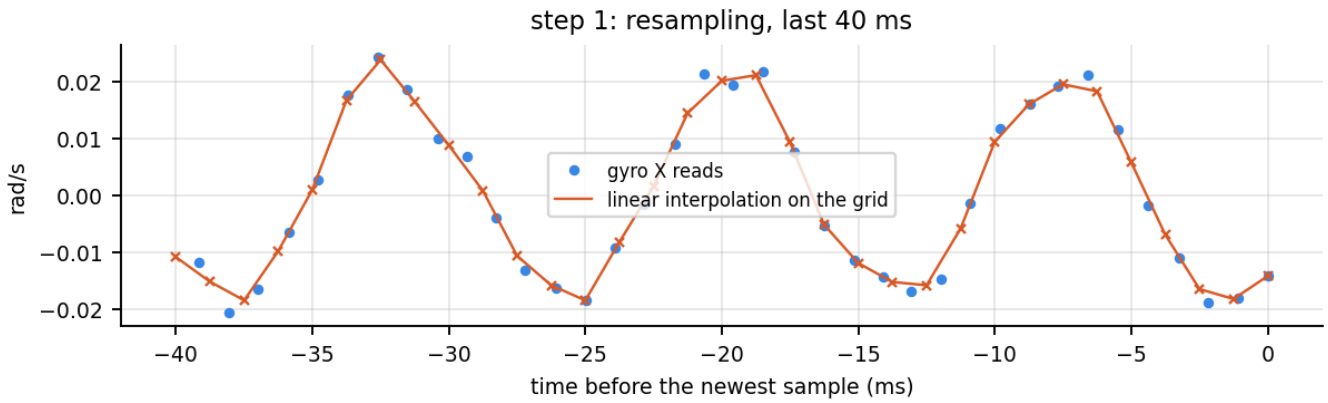
2. Step 1: resampling on an even grid

The driver reads the IMU from a 1 kHz callback that does not fire at even times: on the board the effective rate is 880 to 950 Hz and each sample carries its own read time. An FFT needs evenly spaced samples, so every 250 ms the newest 318.75 ms are interpolated linearly, by their time stamps, on 256 points 1250 µs apart that end at the newest sample:

```
t0      = t_newest - 255 * 1250 us
grid[n] = t0 + n * 1250 us                    n = 0 ... 255
x[n]    = a + (grid[n] - t_a) / (t_b - t_a) * (b - a)
          a, b: the reads just before and just after grid[n]
skip the spectrum if t_b - t_a > 20 ms (a gap in the reads)
```



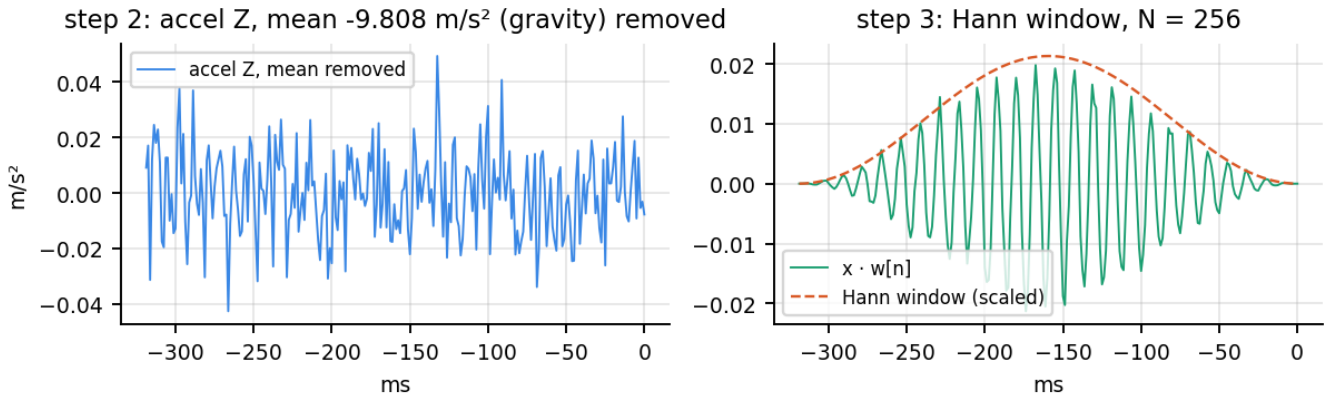




3. Steps 2 and 3: mean removal and window

The mean of each axis is removed (gravity on the accelerometer, the bias on the gyroscope), then the samples are multiplied by a Hann window. The window keeps the energy of a line in a few bins instead of spreading it over the whole spectrum.

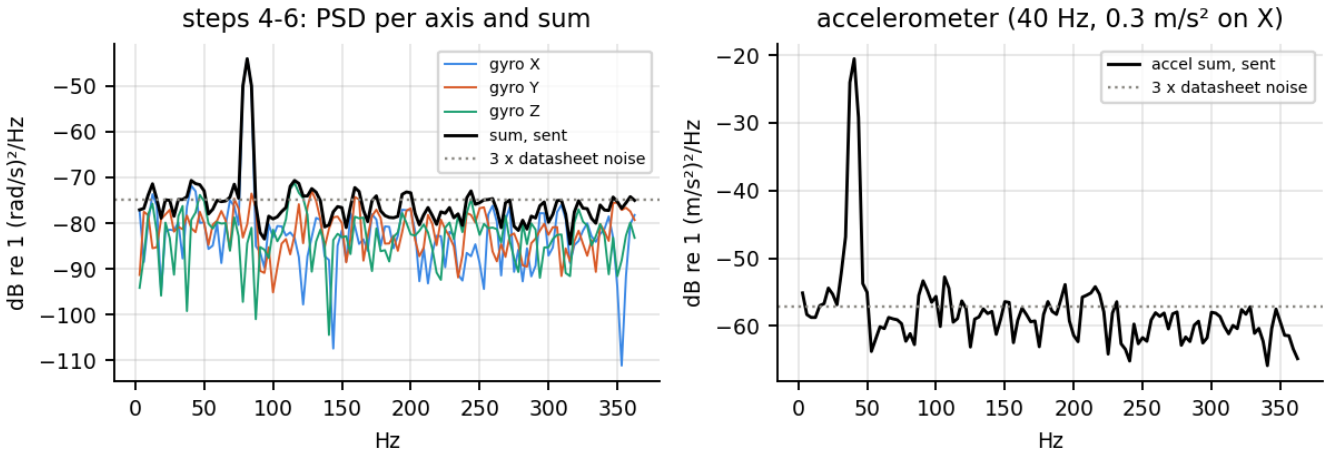
```
w[n]  = 0.5 * (1 - cos(2*pi*n / (N - 1)))     n = 0 ... 255
xw[n] = (x[n] - mean(x)) * w[n]
S2    = sum(w[n]^2) = 95.6250
```



4. Steps 4 to 6: FFT, power spectral density, sum, dB

A 256-point FFT of each axis (imaginary input zero) gives X[k]. The one-sided power spectral density of bins 1 to 116, the sum over the three axes and the conversion to dB:

```
PSD_axis[k] = 2 * |X[k]|^2 / (fs * S2)            k = 1 ... 116
value[i]    = 10 * log10( PSD_x[k] + PSD_y[k] + PSD_z[k] )   i = k - 1
              (power clipped at 1e-20, i.e. -200 dB)
f(i)        = (i + 1) * 3.125 Hz
```



Worked numbers at the gyro line (81.25 Hz is FFT bin k = 26, index i = 25 in the values sent):

Quantity	Value
X[26] of gyro X	-0.2977 -1.1909 j
|X[26]|²	1.5070e+00
scale 2 / (fs · S2)	2 / (800 · 95.6250) = 2.6144e-05
PSD_x[26]	3.9398e-05 (rad/s)²/Hz
PSD_y[26] + PSD_z[26]	1.3081e-08 (rad/s)²/Hz
value[25] sent	10 · log10(3.9411e-05) = -44.04 dB
Check: Σ PSD · df over bins 24..28	1.855e-04 (rad/s)², the line power A²/2 = 2.000e-04
Noise floor, 3 axes, datasheet	10 · log10(3 · 1.047e-04²) = -74.8 dB

The check row shows the normalisation: integrating the density over the bins of a line gives back its power, whatever the window, here within 7 %. The difference is mostly the linear interpolation, which attenuates high frequencies a little (estimated about 4 % in power at 81 Hz with reads 1.1 ms apart).

5. Packing: two messages per sensor

The 116 values of a sensor do not fit in one `data[58]`, so each spectrum is split in two parts. All four messages of an update carry the same `time_usec`, the autopilot time of the newest IMU sample:



Field	Type	Value
time_usec	uint64_t	time of the newest IMU sample, µs since boot; equal in the parts of one spectrum
array_id	uint16_t	2 · sensor + part (sensor 0 gyro, 1 accel; part 0 or 1)
name	char[10]	"FFT_GYRO" or "FFT_ACC", padded with NUL
data	float[58]	data[j] = value[58 · part + j]; frequency (58 · part + j + 1) · 3.125 Hz

The sender emits one message per main-loop cycle, and only when every active link has room for it (it waits at most 100 ms for a full link): the four messages of an update are spread over a few tens of milliseconds. On the CORV USB link, sending them all at once lost the last two.

6. On the wire: a real packet

A gyro part 0 message recorded from the board, encoded again with pymavlink (MAVLink 2, system 1, component 1). In the payload the extension field `data` comes after the base fields:

Payload bytes	Field	Bytes here (little endian)	Decoded
0 - 7	time_usec	62 07 9d 00 00 00 00 00	10291042 µs
8 - 9	array_id	00 00	0 (gyro, part 0)
10 - 19	name	46 46 54 5f 47 59 52 4f 00 00	FFT_GYRO
20 - 23	data[0]	8b 06 9c c2	-78.01 dB at 3.125 Hz
24 - 27	data[1]	96 31 92 c2	-73.10 dB at 6.25 Hz
...			
248 - 251	data[57]	54 f2 df c2	-111.97 dB at 181.25 Hz

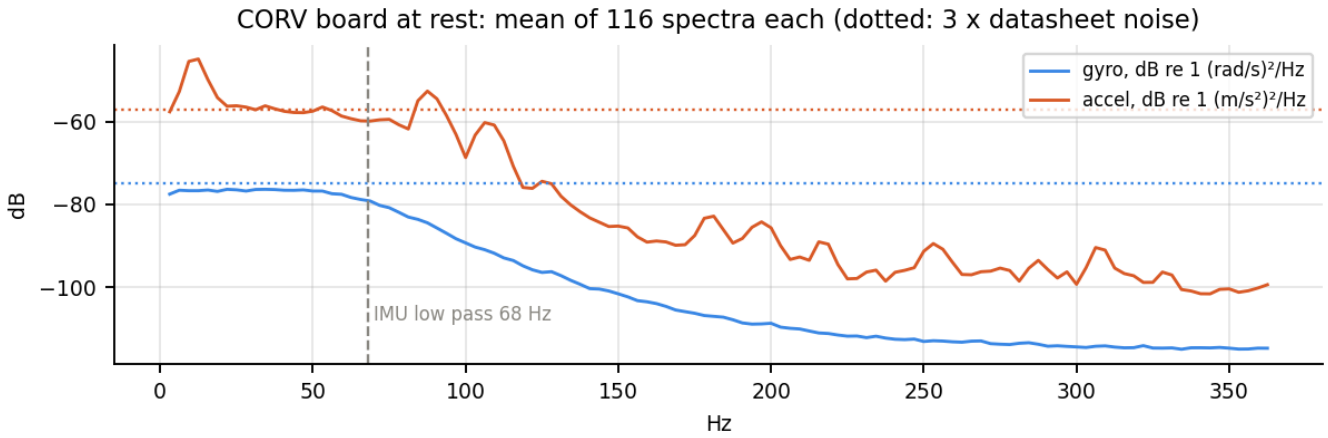
```
header (10 bytes): fd fc 00 00 00 01 01 5e 01 00
  fd = MAVLink 2, payload length 252, msgid 350 = 5e 01 00
payload: 252 bytes, then 2 bytes of CRC (CRC_EXTRA 232)
total on the wire: 264 bytes, about 4.3 kB/s at 4 updates per second
```

7. Receiver: joining the parts

```
on DEBUG_FLOAT_ARRAY msg:
    name   = msg.name up to the first NUL
    sensor = msg.array_id >> 1            # 0 gyro, 1 accel
    part   = msg.array_id & 1
    if sensor > 1 or name != ['FFT_GYRO', 'FFT_ACC'][sensor]: ignore
    p = pending[sensor]
    if p is None or p.time_usec != msg.time_usec:
        p = pending[sensor] = {time_usec, parts: [None, None]}   # an incomplete one is dropped
    p.parts[part] = msg.data[0:58]
    if both parts:
        values = parts[0] + parts[1]       # 116 values, f(i) = (i + 1) * 3.125 Hz
        publish(sensor, time_usec, values); pending[sensor] = None
```

Parts may arrive in either order. A part with a new `time_usec` starts a new spectrum and drops the incomplete one (a message was lost). No message for about 1 s means the stream stopped. Use `time_usec` to join and order, the arrival time for display windows. To get an amplitude: ASD = 10^(value / 20) in rad/s/√Hz or m/s²/√Hz.

8. Real spectra from the CORV board



The board on a bench, 30 s. The gyroscope is the sensor noise: flat up to the IMU low pass at about the datasheet density summed over three axes (measured -77 dB from 5 to 40 Hz, expected -75 dB), then falling with the 4th-order 68 Hz filter. The accelerometer sits at the datasheet level (-56 dB from 5 to 40 Hz, expected -57 dB) but also shows a line near 12.5 Hz and bumps above 68 Hz: something on the bench shook the board (the source was not identified). In flight a motor or a propeller shows as a line at its rotation frequency and its harmonics.

9. Where it is

Part	File
Sender	ardupilot-corv: libraries/AP_HybridPF/AP_HybridPF_Spectrum.{h,cpp}
Format, text	ardupilot-corv: libraries/AP_HybridPF/FFT_SPECTRUM_PROTOCOL.md (also in Corv GCS docs/)
Receiver	Corv GCS: js/core/SpectrumMonitor.js (joining), js/ui/SpectrumViews.js (charts)
Enable	HPF_FFT_ENABLE = 1 on the vehicle; firmware built with HAL_HYBRIDPF_SPECTRUM_ENABLED